EX 7	A simple 2D Tapping Game using Android's Canvas and View Logic

Aim

To design a simple 2D tapping game using Android's Canvas and View logic.

Definitions

2D Game

A 2D game is a game in which characters, objects, and environments move and interact in a two-dimensional space using only height and width.

Tapping Game

A tapping game is a 2D game in which the player earns points or progresses by touching or tapping objects displayed on the screen.

Android Canvas

Canvas is a class in Android that provides methods to draw shapes, text, and images on the screen.

Android View Logic

Android View logic is the programming logic written inside a View class to control drawing, measuring, layout, and user interaction.

Procedure

1. Open Android Studio IDE → File → New → New Project → specify the application name "GameApp" and company domain "com.mad.tapgame" → click "next" → choose Minimum SDK "API 17:Android 4.2(Jelly Bean)" → click "Next" → choose "Blank Activity" → click "next" → specify the Activity Name "MainActivity" → click "Finish".
2. Open MainActivity.java under app/java/ tapgame.mad.com.gameapp and type the following codes:

MainActivity.java

```
package tapgame.mad.com.gameapp;

    import android.app.Activity;
    import android.os.Bundle;
    import android.widget.TextView;

public class MainActivity extends Activity {

  private TextView txtScore;
  private GameView gameView;

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    txtScore = (TextView) findViewById(R.id.txtScore);
    gameView = (GameView) findViewById(R.id.gameView);

    gameView.setScoreListener(new GameView.ScoreListener() {
      @Override
      public void onScoreChanged(int score) {
        txtScore.setText("Score: " + score);
      }
    });
  }
}
```

3. Right click on tapgame.mad.com.gameapp package → New → Java Class → Name: GameView → ok. Type the following codes in GameView.java,

GameView.java

```
package tapgame.mad.com.gameapp;

    import android.content.Context;
    import android.graphics.Canvas;
    import android.graphics.Color;
    import android.graphics.Paint;
```

```java
    import android.util.AttributeSet;
    import android.view.MotionEvent;
    import android.view.View;

    import java.util.Random;

public class GameView extends View {

    private Paint paint;
    private float circleX, circleY;
    private float radius = 100;
    private int score = 0;
    private Random random;

    // Score Listener Interface
    public interface ScoreListener {
        void onScoreChanged(int score);
    }

    private ScoreListener scoreListener;

    public void setScoreListener(ScoreListener listener) {
        this.scoreListener = listener;
    }

    // Constructors (required for XML)
    public GameView(Context context) {
        super(context);
        init();
    }

    public GameView(Context context, AttributeSet attrs) {
        super(context, attrs);
        init();
    }

    public GameView(Context context, AttributeSet attrs, int defStyleAttr) {
        super(context, attrs, defStyleAttr);
        init();
    }

    private void init() {
        paint = new Paint();
        paint.setColor(Color.RED);
        paint.setAntiAlias(true);
        random = new Random();
    }

    @Override
    protected void onDraw(Canvas canvas) {
```

```
        super.onDraw(canvas);

        if (circleX == 0 && circleY == 0) {
            resetCircle();
        }

        canvas.drawCircle(circleX, circleY, radius, paint);
    }

    private void resetCircle() {
        circleX = radius + random.nextInt(getWidth() - (int)(2 * radius));
        circleY = radius + random.nextInt(getHeight() - (int)(2 * radius));
    }

    @Override
    public boolean onTouchEvent(MotionEvent event) {

        if (event.getAction() == MotionEvent.ACTION_DOWN) {

            float touchX = event.getX();
            float touchY = event.getY();

            double distance = Math.sqrt(
                    Math.pow(touchX - circleX, 2) +
                            Math.pow(touchY - circleY, 2)
            );

            if (distance <= radius) {
                score++;

                if (scoreListener != null) {
                    scoreListener.onScoreChanged(score);
                }

                resetCircle();
                invalidate();
            }
        }

        return true;
    }
}
```

4. Open activity_main.xml under app/res/layout and type the following codes:

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

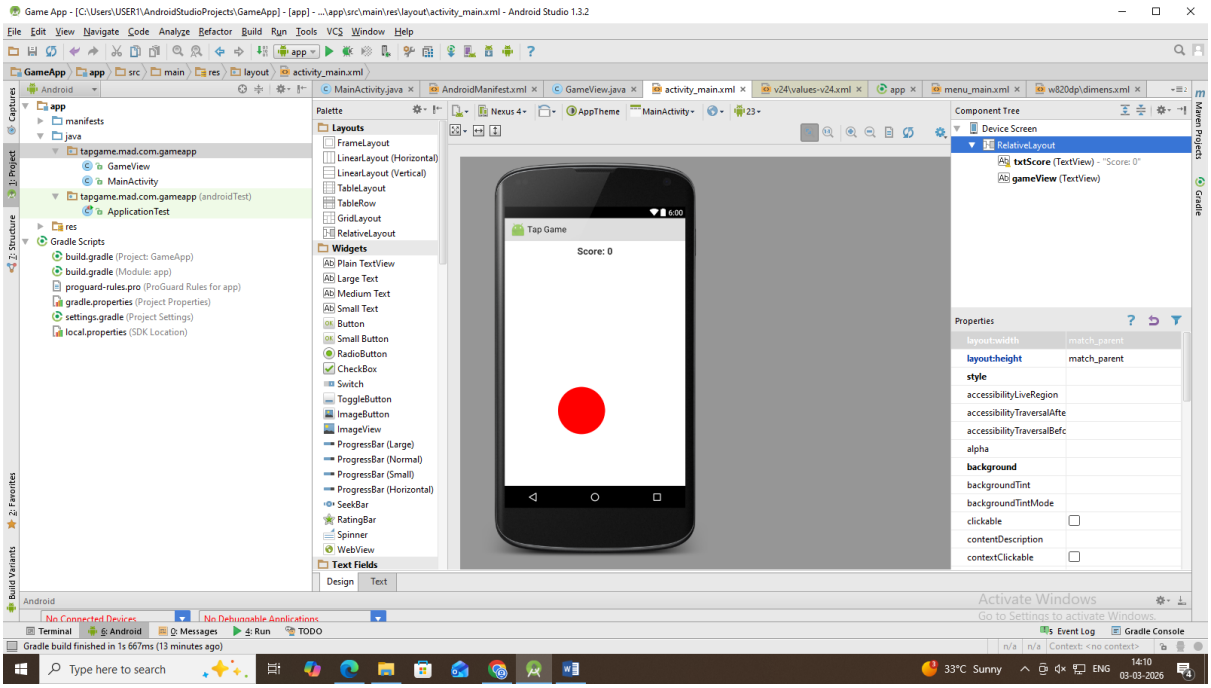
```
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/txtScore"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Score: 0"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:padding="10dp" />

    <tapgame.mad.com.gameapp.GameView
        android:id="@+id/gameView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_below="@id/txtScore" />

</RelativeLayout>
```

5. The design of the application will be as follows:



6. Open AndroidManifest.xml under app/manifests folder and do the following changes:

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    package="tapgame.mad.com.gameapp"
    android:versionCode="1"
    android:versionName="1.0">

    <uses-sdk
        android:minSdkVersion="17"
        android:targetSdkVersion="17" />

    <application
        android:theme="@style/AppTheme"
        android:label="Tap Game"
        android:icon="@mipmap/ic_launcher">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>
</manifest>
```

7. Open build.gradle (Module: app) under app/Gradle Scripts and do the following changes:

   build.gradle

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 17
    buildToolsVersion "23.0.3"

    defaultConfig {
        applicationId "tapgame.mad.com.gameapp"
        minSdkVersion 17
        targetSdkVersion 17
        versionCode 1
        versionName "1.0"
    }
}
```

8. Open styles.xml under app/re/values folder and do the following changes:

   styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
```

```
    <style name="AppTheme" parent="android:Theme.Holo.Light">
    </style>

</resources>
```

9. Open menu_main.xml under app/res/menu folder and do the following changes:

   menu_main.xml

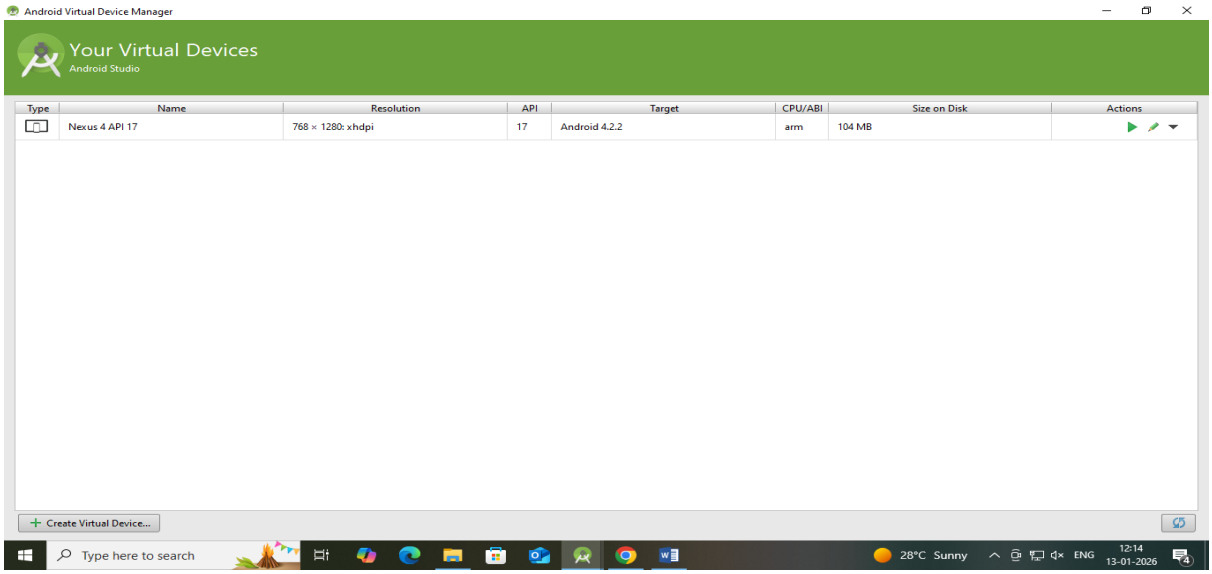
```
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/action_settings"
        android:title="Settings"
        android:orderInCategory="100"
        android:showAsAction="never" />

</menu>
```

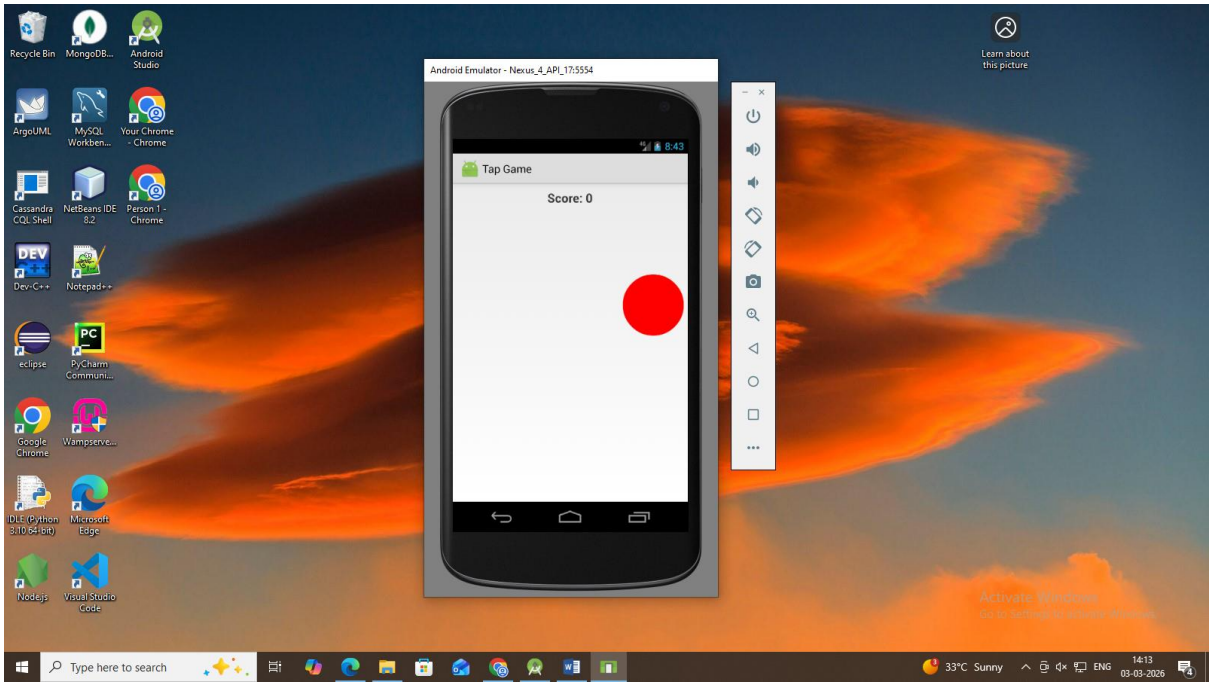
10. Go to Tools → android → AVD Manager → click "+ create a virtual device" → select "phone" from category → select "Nexus 4" from the list → click "next" → select Release name: Jelly Bean, API Level: 17, ABI: armeabi-v7a, Target: Android 4.2.2 from the list → click "next" → Choose orientation "portrait" → click "finish".

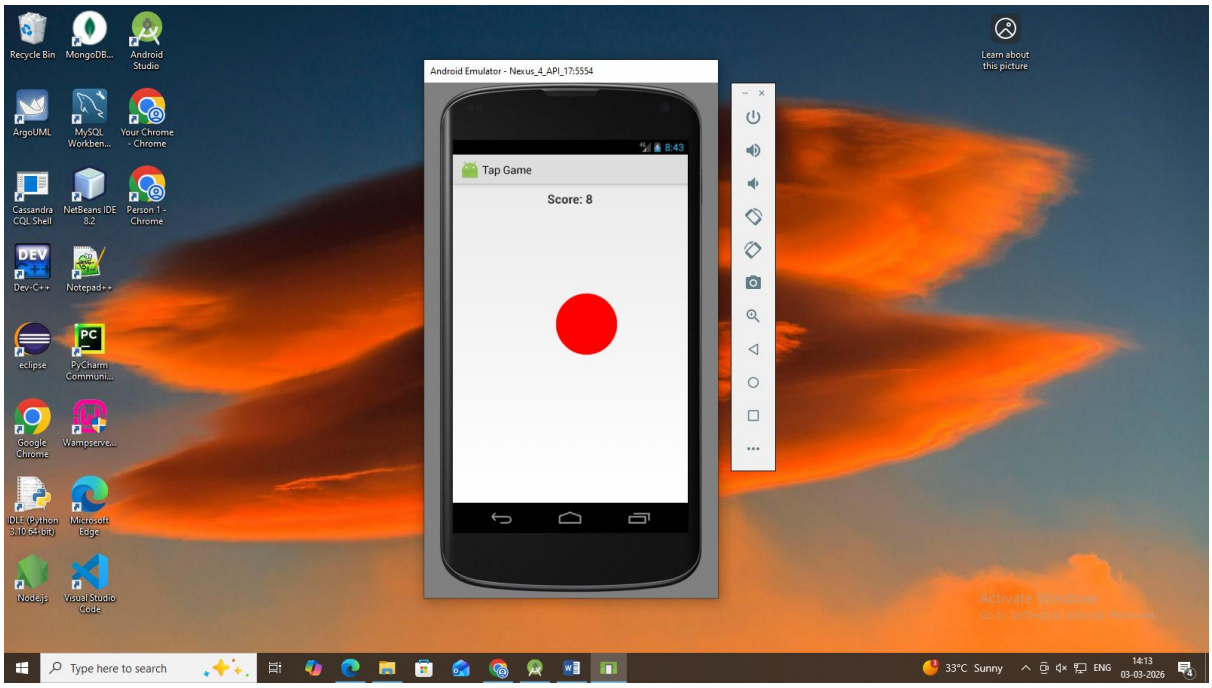
The following window will appear after configuring AVD:



11. Click "Run app" button in the Android Studio → choose android virtual device → click "ok".

Output


Android Emulator - Nexus_4_API_17:5554
Tap Game
Score: 0


Android Emulator - Nexus_4_API_17:5554
Tap Game
Score: 8

Result

Thus, a simple 2D tapping game using Android's Canvas and View logic has been designed.